

Similarity-Based Secure Deduplication for IIoT Cloud Management System

Yuan Gao^{ID}, Graduate Student Member, IEEE, Liquan Chen^{ID}, Senior Member, IEEE, Jinguang Han^{ID}, Senior Member, IEEE, Shui Yu^{ID}, Fellow, IEEE, and Huiyu Fang^{ID}, Student Member, IEEE

Abstract—With the development of the Industrial Internet of Things (IIoT), the scale of IIoT data is rapidly increasing, bringing significant challenges to existing data management systems. To tackle this issue, we propose a similarity-based secure deduplication for IIoT cloud management system, which can effectively balance the security and availability of IIoT data and minimize the storage cost. Concretely, we propose a similarity-based secure deduplication algorithm for IIoT (IIoT-SBSD) by designing a similarity-preserving tag (IIoT-Simhash). This algorithm can perform similarity deduplication over ciphertexts, thus reducing storage space while ensuring data security. Besides, we construct a parallelizable edge-based deduplication framework in which similarity comparison and deduplication operations are performed directly by edge nodes, significantly alleviating the transmission pressure. Additionally, we propose similarity-based proofs of ownership, S-PoWs, to mitigate the impact of data deduplication on the user's access to the IIoT data. Experimental results show that our method significantly reduces storage space and transmission bandwidth without increasing the computation burden.

Index Terms—Data similarity comparison, deduplication, edge computing, industrial Internet of Things, proofs of ownership.

I. INTRODUCTION

WITH the development of Internet of Things (IoT) technology, various IoT systems and infrastructures have been widely deployed, especially in the Industrial Internet of Things (IIoT) field [1]. According to a survey from *IoT Analytics* [2], more than 75 billion IoT devices will be connected to the Internet up to 2025. Massive IoT devices generate a large amount of data, much of which is highly repetitive due to the cyclical and collaborative nature of IoT devices working [3]. The massive and highly repetitive IoT data has become an enormous

problem to solve in data storage and management, and even getting increasingly urgent with the rapid development of IoT.

The issue of limited storage space in IIoT has prompted the proposal of two distinct approaches: iterative storage and transfer storage. The iterative overlay approaches involve defining a “data shelf life” and subsequently deleting or overwriting IIoT data that surpasses this limit. However, such methods do not consider data utility and may compromise the availability and traceability of IIoT data, as noted in Weber et al.'s work [4]. Transfer storage can be further subdivided into edge node storage [5] and Cloud Service Provider (CSP) storage [6], [7], [8]. The advantage of edge storage is that data can be processed and stored closer to the user, thus reducing network latency and bandwidth consumption. However, its storage capacity and processing capability are limited by the hardware performance of the edge nodes, which may not be able to handle large-scale or complex data. Compared to edge storage, cloud storage requires data transmission over the Internet and thus may have relatively high network latency and bandwidth consumption, but it can provide larger storage capacity and support large-scale and complex data processing tasks. Transfer storage IIoT data can significantly alleviate storage pressure on IIoT devices. However, due to the cyclical nature of IIoT device work, a considerable amount of redundant information is produced, leading to a significant reduction in cloud storage efficiency. To address this issue, various data deduplication technologies [9] have been proposed and implemented in actual CSPs [10], [11], such as Amazon S3, Bitcasa, and Microsoft Azure [12].

Different from ordinary Internet data, a significant characteristic of most IIoT data is that they are highly structured, i.e., with fixed length and data type (as shown in Fig. 1). For ease of presentation, IIoT data are classified into two types: The data produced when problems with IIoT system arise is *abnormal data*, and the other data is *regular data*. Fig. 1(a) and (b) present examples of regular data, and Fig. 1(c) presents an example of abnormal data. As shown in these figures, each IIoT data record mainly consists of two parts: one involves device information and time stamps (green part), such as device location, data collection time, and anomaly data identification, which are sensitive; the other part is the data record (blue part), such as device temperature, product quantity, and device noise, which are the main body of IIoT data. Most IIoT data records are highly similar due to the fact that these IIoT data are generated by IoT sensors of the same type, and the tiny difference of these similar data comes from the sensors' different working environments,

Manuscript received 8 November 2022; revised 7 May 2023; accepted 2 August 2023. Date of publication 7 August 2023; date of current version 11 July 2024. This work was supported in part by the National Natural Science Foundation of China under Grants U22B2026 and 62002058, in part by the National Key Research and Development Program of China under Grant 2020YFE0200600, and in part by the Natural Science Foundation of Jiangsu Province under Grant BK20200391. (Corresponding author: Liquan Chen.)

Yuan Gao, Jinguang Han, and Huiyu Fang are with the School of Cyber Science and Engineering, Southeast University, Nanjing 210096, China (e-mail: 230209298@seu.edu.cn; jghan@seu.edu.cn; fhy837637215@gmail.com).

Liquan Chen is with the School of Cyber Science and Engineering, Southeast University, Nanjing 210096, China, and also with the Purple Mountain Laboratories for Network and Communication Security, Nanjing 211111, China (e-mail: lqchen@seu.edu.cn).

Shui Yu is with the School of Computer Science, University of Technology Sydney, Ultimo, NSW 2007, Australia (e-mail: shui.yu@uts.edu.au).

Digital Object Identifier 10.1109/TDSC.2023.3302891

IIoT Data Record	
Exception flag:	0 (bool)
Temperature:	80.2 °C (float)
Quantity:	120 /min (int)
Device noise:	85 dB (int)
.....	
Address:	Springfield (string)
Timestamp:	1989/12/17 (datetime)
Device ID:	Doh-7-G (string)

(a)

IIoT Data Record	
Exception flag:	0 (bool)
Temperature:	80.3 °C (float)
Quantity:	120 /min (int)
Device noise:	85 dB (int)
.....	
Address:	Springfield (string)
Timestamp:	1989/12/18 (datetime)
Device ID:	Doh-7-G (string)

(b)

IIoT Data Record	
Exception flag:	1 (bool)
Temperature:	153.7 °C (float)
Quantity:	36 /min (int)
Device noise:	102 dB (int)
.....	
Address:	Springfield (string)
Timestamp:	1989/12/19 (datetime)
Device ID:	Doh-7-G (string)

(c)

Fig. 1. Three examples of IIoT data records: (a) and (b) are regular data records, and (c) is an abnormal data record. The data types are also indicated in parentheses.

in time, in space, or both. Unfortunately, existing deduplication methods can not be applied to similar IIoT data. Besides, IIoT data is often stored in the cloud by its owners in an encrypted manner to ensure data security. The different encryption methods adopted by these owners destroy the similarity of the data and thus greatly increase the difficulty of similar data deduplication. In that sense, an encrypted data based similarity deduplication scheme is highly desirable, which could achieve efficient IIoT data storage while avoiding uploading the unencrypted raw data to the CSPs. It is worth noting that an important aspect of IIoT data deduplication lies in maintaining user ownership of the data [13], [14]. Therefore, in similar deduplication schemes, *Proofs of Ownership (PoWs)* should also be focused.

To tackle these issues, we propose a similarity-based secure deduplication for the cloud management system, aiming to resolve the conflict between CSP data management and IIoT data storage requirements. Concretely, we devise IIoT-SBSD, a similarity-based secure deduplication algorithm that is backed up by the fuzzy extractor and pseudorandom generator. IIoT-SBSD enables ciphertext-based data similarity comparison and similar data deduplication, thus significantly reducing storage costs. We also take data accessibility into account and propose similarity-based proofs of ownership, S-PoWs, which improve data security by initiating similarity-based challenges to verify user access to data. Furthermore, a parallelizable edge-based data processing approach is proposed, which can perform data deduplication before uploading to the cloud, thus boosting the efficiency of data computation while alleviating transmission pressure. Specifically, the contributions of this paper are as follows:

- A similarity-based secure deduplication for IIoT cloud management system is proposed for the first time, which can provide efficient management of IIoT data while ensuring data security.
- We propose IIoT-SBSD, a similarity-based secure deduplication algorithm for IIoT, which enables ciphertext similarity comparison and similar data deduplication, thus significantly reducing storage costs.

- We design a similarity-based proofs of ownership, S-PoWs, to avoid the impact of data deduplication on the user's access to the IIoT data.
- We conduct comprehensive experiments on several real-life datasets, and the results demonstrate that our approach can significantly reduce storage space and transmission bandwidth without increasing computing overhead.

The remainder of this paper is organized as follows. We review the related works in Section II. Section III presents an introduction to preliminaries. Section IV describes the system architecture, the system design, and the related algorithm presented in it. In Section V, we analyze the security of the system. The experimental setting and results are detailed in Section VI, followed by a summary section.

II. RELATED WORK

In this section, we review the literature related to our scheme.

Secure Data Deduplication. Secure data storage has received widespread attention due to the growing problem of information security. To ensure data security, data owners often encrypt data before uploading it to CSPs. However, this brings difficulties for data deduplication since the same data will be encrypted into different ciphertexts due to the different data encryption approaches and encryption keys their owners may choose. To tackle this issue, scholars have conducted several types of research. For example, Jayapandian et al. [23] proposed the convergent encryption (CE) system which enables the same plaintext to be turned into the same ciphertext by using the file hash value as the encryption key and thus achieves ciphertext deduplication. Based on CE, Bellare et al. [24] introduced the concept of message-locked encryption (MLE) which enables the cloud server to determine whether two ciphertexts correspond to the same plaintext. However, it is possible for an adversary with background knowledge of the ciphertext to infer the plaintext data through exhaustive encryption and comparison. To enhance the security of deduplication, Puzio et al. [15] proposed PerfectDedup, a novel scheme that takes into account

the popularity of data segments and employs Perfect Hashing to ensure block-level deduplication and data confidentiality simultaneously. This approach minimizes client-side overhead and outsources the main computational load to the cloud storage provider. Nevertheless, these methods are vulnerable to offline exhaustive assault attacks [25] and cannot satisfy semantic security requirements.

To address these challenges, Wang et al. [26] presented a key-sharing method based on proof of ownership for secure deduplication. In this method, only the initial uploader of the data owner encrypts the data with a randomly selected convergent key (CK) and distributes it in the cloud. Only users who have access to the claimed data are able to retrieve the CK. The CK just needs to store a single duplicate piece of data once. Additionally, a multi-user updatable encryption scheme was proposed [27]. By sending an update token to the cloud, the data owner can specifically update the distant ciphertext under a new group key. This approach has led to the development of a new secure deduplication technique that effectively ensures storage data security by preventing unauthorized user access. Furthermore, to update the data authority, the data owner can send a token to the cloud, thereby saving communication and computation expenses.

In addition, there is some literature on different research directions. Yuan et al. [17] presented a blockchain-based approach for public auditing and secure deduplication with fair arbitration. Through public regulation and compensation incentives, this scheme effectively reduces the computational cost of tag verification and data storage cost. Furthermore, Tian et al. [18] proposed a blockchain-based system for secure deduplication and shared auditing in decentralized storage which achieves effective space-saving and avoids data loss caused by single-point failure and duplicate-faking attacks. To achieve decentralized public auditing without a third-party auditor, this scheme also utilizes a blockchain-based two-way shared auditing mechanism where users exchange the audit authenticators and results of outsourced data among themselves to prevent tedious audit duties. What's more, an effective and feature-rich attribute-based encryption (ABE)-based Metaverse healthcare data sharing method [28] is proposed which realizes consistent encryption computation overhead according to a multiserver topology and takes into account how the data will be used after deduplication. The Metaverse healthcare ciphertext validity and equivalence detection are also offered in this technique to reduce the number of invalid ciphertexts and accomplish secure deduplication. Additionally, a brand-new attribute-based re-encryption is suggested for delegation of authority following deduplication. However, all of the aforementioned methods only consider deduplication of the same data and thus cannot be applied to IIoT scenarios.

Similar Data Deduplication. In numerous practical contexts, data obtained from the Industrial Internet of Things may bear a resemblance and convey the same type of information. Such a large amount of similar data can significantly encumber the storage capacity. Consequently, several systems have been proposed to address the challenges of similar data retrieval and deduplication. Manku et al. [29] first proposed a similar data deduplication algorithm based on the Simhash algorithm. It generates similarity-preserving tags using the Simhash method

and decreases the dimensionality of text, enabling rapid comparison of online pages and resolving the issue of deduplication of billions of web pages. Based on this, Sood et al. [30] developed a dimensionality-reduction method to find related document pairs in huge collections. Compared to Manku's scheme, this one can be much faster and needs less storage space. It cleared the way for the probabilistic search method based on the Simhash Hamming space. Wu et al. [31] provided an Error Correcting Code (ECC) assisted deduplication approach, called EaD. Data similarity is initially determined by EaD using the fingerprints of data chunks represented by their ECC values, greatly lowering the need for expensive cryptographic hash computation and minimizing the memory space overhead. Based on the results of the identification, similar data chunks and associated ECCs are retrieved from the flash in order to carry out a byte-by-byte comparison in memory to successfully identify and eliminate similar data chunks. Additionally, Talasila et al. [32] developed generalized deduplication, in which similar data is systematically deduplicated by first transforming chunks of each file into two parts: a basis and a deviation. This method characterizes the generalized deduplication performance by employing Golomb-Rice codes as an appropriate data transform function to find commonalities among all files kept in the system. In theory and simulations, generalized deduplication can use 300 times less storage space than conventional deduplication.

The aforementioned method places greater emphasis on deduplicating plaintext similarities rather than ciphertext data. Liu et al. [33] proposed a secure approach to ciphertext deduplication based on data similarity, which leverages both bloom filter and content-defined chunking technologies. This method effectively identifies data similarities and performs block-level identical ciphertext deduplication. Nevertheless, it neglects to account for the significance of data integrity and availability. To address this issue, a straightforward solution would be to propose a secure similarity-based data deduplication approach that ensures both data integrity and availability. Zhang et al. [21] proposed a privacy-preserving and compression-based data deduplication system in a fog-cloud network, supporting lossless deduplication of similar data in the encrypted domain. To achieve secure deduplication over similar data, the authors utilized generalized deduplication techniques and cryptographic primitives, and devised a two-level deduplication protocol for secure and efficient deduplication. This system is resistant to brute-force and frequency analysis attacks and ensures that only the data operator can access the message equality information. Recently, Jiang et al. [22] designed a novel deduplication system, named Fuzzy Deduplication (FuzzyDedup), to implement secure deduplication of similar data. This method achieves ciphertext-based deduplication for related data by utilizing a fuzzy extractor to create a fuzzy-style deduplication encryption.

Table I shows a functionality comparison of several related deduplication approaches. As shown in Table I, the existing deduplication methods are not suitable for IIoT scenarios due to their mainly considering the secure deduplication of the same data. Despite the fact that the scheme [22] considers data security and data availability in similar data deduplication procedures, this scheme performs centralized deduplication operations for all duplicate data by the CSP, which limits its applicability on

TABLE I
COMPARISON OF FUNCTIONALITY

Scheme	Deduplication level	Deduplication position	Focus	Data security	Similarity deduplication	Availability
[15]	Block level	Cloud	Secure data deduplication	✓	×	×
[16]	Block level	Local	Secure data deduplication	✓	×	×
[17]	Block level	Local	Secure data deduplication	✓	×	✓
[18]	Block level	Local	Secure data deduplication	✓	×	✓
[19]	File level	Edge node	Efficient deduplication	×	✓	✓
[20]	File level	Edge node & Cloud	Cross-domain deduplication	✓	×	×
[21]	File level	Fog node & Cloud	Similar data deduplication	✓	✓	×
[22]	File level	Cloud	Similar data deduplication	✓	✓	✓
Ours	File level	Edge node	Similar data deduplication	✓	✓	✓

large-scale IIoT data. To solve this problem, this paper proposes a system for this special application scenario, which can achieve similarity-based secure deduplication. More importantly, our approach utilizes edge-based settings to perform parallelizable deduplicate and adopts access permission expansion to improve IIoT data availability. In summary, our system not only creates a similarity-based secure deduplication algorithm but also implements a distributed IIoT data management system and achieves a certain level of security.

Challenges: To construct our scheme, the following problems must be considered.

- As IIoT data has high timeliness and colossal volume, how to increase data processing effectiveness and decrease transmission overhead is a challenging problem.
- How to ensure similar data remain similar after encryption and break the limitations of existing encryption schemes is the problem that should be solved for similarity data deduplication.
- IIoT data storage is designed to serve data analytics, thus maintaining its availability is the primary aspect of data storage to consider. Existing data management systems tend to focus on data security and data storage costs, with insufficient attention to data availability maintenance, making it urgent to propose a data management system that can maintain data availability.

Techniques. The corresponding techniques applied to solve the above problems are as follows.

- To solve the first issue, we add edge nodes in the system to process IIoT data, and parallel operations to handle the real-time demand of IoT data. Additionally, we perform data deduplication operations on edge nodes and only upload the valuable data to CSP, which reduces communication overhead and saves storage space.
- A similarity data ciphertext deduplication technique for IIoT (IIoT-SBSD) is presented to address the second issue. By generating the similarity-preserving tag algorithm (IIoT-Simhash), edge nodes can compare the similarity of ciphertext, and combine the characteristics of IoT data to deduplicate similar data only for the regular data.
- To address the third issue, we design a similarity-based proofs of ownership (S-PoWs) to implement data ownership verification. When a user initiates a data query request, CSP will make an S-PoWs challenge to this user, and only those who pass the challenge can get access to the query data, thus improving data security while ensuring data availability.

III. PRELIMINARIES

A. Simhash

Simhash [29] is a feature fingerprinting method that generates a compact representation of multi-dimensional vectors via random mapping. Its distinguishing feature is that the cosine similarity between the respective feature vectors and the Hamming distance of two document fingerprints are positively associated. It is currently mainly used in Google web page deduplication work. In a nutshell, the Simhash algorithm's primary tasks include downscaling the text, generating a fingerprint, and comparing the Simhash values of other texts in terms of Hamming distance to ascertain how similar the two texts are.

The SimHash algorithm has five main processes: Separate, Hash, Weight, Merge, and Dimension reduction.

- 1) *Separate*: The given text is separated into n feature words, and each word is assigned a weight.
- 2) *Hash*: Determine whether the feature word is numeric data; if not, use the hash function to convert each word vector into an n -bit binary string.
- 3) *Weight*: The weight vector is calculated by converting the bits in the hash binary string that are 1 to the positive feature word weight value and 0 to the negative feature word weight value.
- 4) *Merger*: The weight vectors of each feature word in a text are calculated, and then they are added together to create a weight vector of the text.
- 5) *Dimension reduction*: To calculate the text's Simhash value, the weight vector that was created after merging is processed so that bits greater than 0 are set to 1 and bits less than 0 to 0.

In contrast to typical hashes, Simhash provides users with additional details regarding the degree of variances between unequal original contents and whether the original contents are equal. Simhash is a type of Locality-Sensitive Hashing (LSH), and it has properties that increase the probability of data similarity as the Simhash value approaches zero. This provides a lot of conveniences for the discrimination of data similarity.

B. Fuzzy Extractor

The Fuzzy Extractor (FE) [34], which permits some noise (or error) in the input, can extract the same uniform random string as long as the inputs are similar. It is generally built as Fig. 2.

The Fuzzy Extractor $FE = (Gen, Rep)$ is composed of two algorithms in particular: Generation algorithm Gen and Regeneration algorithm Rep :

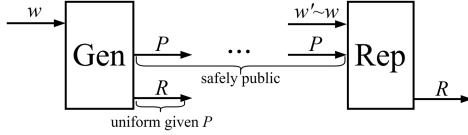


Fig. 2. The construction of fuzzy extractor.

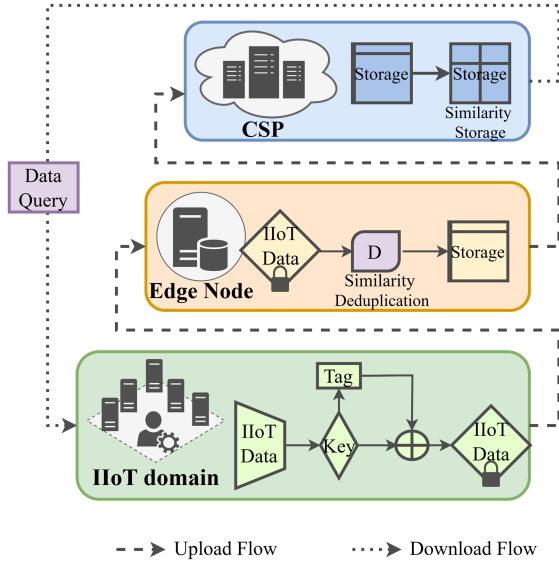


Fig. 3. The cloud management system model includes data encryption and upload, similarity-based secure deduplication, cloud storage management, and data query.

$Gen \rightarrow (P, R)$: Generation algorithm Gen inputs a string w (one sample of a random noise source), yields a string R and an open help string P .

$Rep(w', P) \rightarrow R'$: Regeneration algorithm Rep inputs w' (another sample of a noisy random source) and an open help string P and outputs a string R' .

As explained below, the Fuzzy Extractor exhibits both correctness and security.

Correctness: If $dis(w, w') \leq t$, then $R' = R$, which means that R can be precisely regenerated.

Security: The distribution of R is nearly uniform, and the auxiliary data P do not expose too much about R .

IV. SYSTEM ARCHITECTURE

In this paper, a similarity-based secure deduplication for IIoT cloud management system is proposed. By using the similarity-preserving tag algorithm (IIoT-Simhash), this approach can directly compare ciphertext similarity to achieve similarity-based secure deduplication. We introduce edge computing for IIoT data processing, so edge nodes only need to deliver valid data that has been deduplicated to CSPs, which significantly reduces the transmission overhead. In addition, to avoid the impact of deduplication on data availability, we also design a similarity-based proofs of ownership (S-PoWs) to implement data ownership verification and further improve data security. Fig. 3 shows the system model.

This system mainly consists of three parts: IIoT domain, Edge Node, and CSP.

- **IIoT domain:** As the fundamental unit of industrial IoT, the IIoT domain comprises mostly sensors and administrators. Each IIoT domain generally contains many IoT sensors and produces a lot of IIoT data. The IIoT domain administrator is responsible for encrypting the IIoT data and uploading the encrypted data to the nearest edge node. In this paper, we equate the IIoT domain administrator with the data owner.
- **Edge Node:** The edge node is used to perform data similarity deduplication on the received data as well as temporary maintenance of the deduplicated data and its auxiliary parameters. The edge nodes periodically transmit deduplicated data to the CSP.
- **Cloud Service Provider (CSP):** The CSP represents a semi-trusted entity in charge of managing and storing IIoT data as well as validating user's data ownership, and providing a data query service for users who pass the verification.

More information on how the three entities interact is provided in the following subsections. It's worth noting that, in accordance with the IIoT data characteristics, we only perform deduplication operations on the IIoT data record part of the regular data, which will not be addressed separately in the following.

A. System Design

Fig. 4 depicts the interaction of the system. First, the data owner utilizes the IIoT-Simhash algorithm to generate the IIoT record part's similarity-preserving tag t_w and sends it to the edge node, which performs similarity detection on the data. Edge nodes use the similarity-preserving tag t_w to do similarity detection for regular data. If there has no similar data present, the data owner needs to generate a similar data key k_w and its auxiliary information P_w of data w using a fuzzy extraction technique, then send the ciphertext c_w and auxiliary information P_w to the edge node. On the other hand, if similar data is present at the edge node, the edge node will return the auxiliary information P_w . The data owner will recover the key k_w and use it to encrypt the data for transmission. Eventually, edge nodes perform ciphertext-based deduplication and periodically deliver deduplicated data to CSP. To make future viewing and analysis of the data easier, the CSP compares the received data with the data that already exists in the database and stores similar data via similarity association. When the data owner needs to query the data, they must confirm their ownership with the CSP. If the ownership proof is successful, the data owner can access its own ciphertext data. The specific calculation procedures for t_w , k_w , P_w and c_w etc. are described in the following subsections.

B. IIoT-Simhash: The Similarity-Preserving Tag Algorithm

The format and textual content of IIoT data from the same region and series of devices are very similar, with only minor differences in numeric data (e.g., the blue part in Fig. 1). The traditional Simhash works well for full-text similar data detection, it normally uses a sensitive hash for feature words and treats all feature words as strings. However, this method cannot accurately

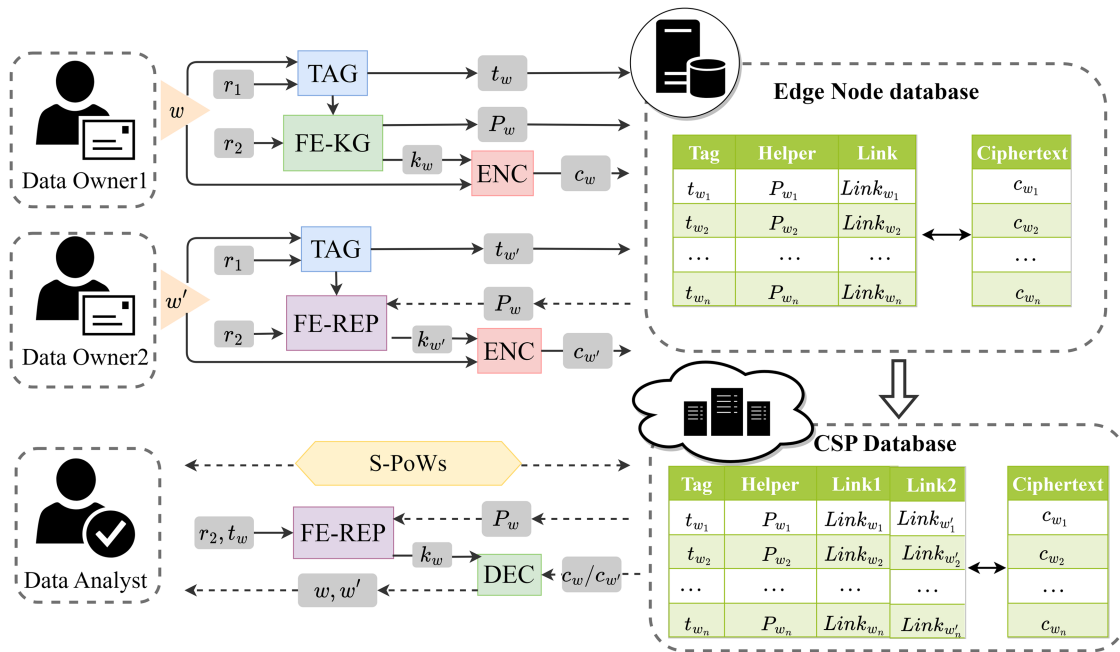


Fig. 4. The interaction diagram of entities in the cloud management system, in which the data owner and data analyst are equated to the IIoT domain.

TABLE II
SIMILARITY COMPARISON OF DIFFERENT TAG ALGORITHMS

[illegible]

distinguish the similarity degree between two similar values and cannot satisfy the special characteristics of IIoT data. We cannot accurately determine the similarity between two IIoT data if we use traditional Simhash to generate similarity-preserving tags. Table II displays the details. To overcome Simhash’s limitations of being insensitive to short texts containing numerical values, we offer the IIoT-Simhash algorithm based on Simhash. This algorithm can accurately determine the similarity between two data through similarity-preserving tags.

The IIoT-Simhash algorithm has six main processes (Separate, Hash, Binarization transformation, Weight, Merger, and Dimension reduction):

- 1) *Separate*: The given data is separated into n feature words, and each word is assigned a weight.
- 2) *Hash*: Determine whether the feature word is numeric data; if not, use the hash function to convert each word vector into an n -bit binary string.
- 3) *Binarization transformation*: Determine whether the feature word is numeric data; if so, binarize the data to replace its hash.

- 4) *Weight*: The weight vector is calculated by converting the bits in the hash binary string that are 1 to the positive feature word weight value and 0 to the negative feature word weight value.
- 5) *Merger*: The weight vectors of each feature word in a text are calculated, and then they are added together to create a weight vector of the text.
- 6) *Dimension reduction*: To calculate the data's similarity-preserving tag, the weight vector that was created after merging is processed so that bits greater than 0 are set to 1 and bits less than 0 to 0.

The exact algorithm is shown in Algorithm 1. First, separate the data into n feature words, denoted as $F(u)$. Second, determine whether each feature word is numeric data; if not, use the hash function to convert each word vector into an n -bit binary string; if so, binarize the data to replace its hash. Subsequently, we perform a weighting process for each binary string. The weight vector is calculated by converting the bits in the hash binary string that are 1 to the positive feature word weight value and 0 to the negative feature word weight value. Finally, all

Algorithm 1: IIoT-Simhash(w).

Input: $W \leftarrow$ array of b zeros
Output: Array $B[1 \dots b]$, the IIoT-Simhash of W

for $i \in F(u)$ **do**
 if $\text{len}(\text{str2num}(i)) == 0$ **then**
 $\phi_i \leftarrow \text{UniformHash}(i)$ **#Compute b-bit hash**
 else
 $\phi_i \leftarrow \text{Denary2Binary}(i)$ **#Convert to b-bit binary**
 for $j = 1$ **to** b **do**
 if $\phi_{ij} == 1$ **then**
 $W[j] \leftarrow W[j] + w_i$
 else
 $W[j] \leftarrow W[j] - w_i$
 end
 end
 end
for $j = 1$ **to** b **do**
 if $W[j] > 0$ **then**
 $B[j] \leftarrow 1$ **#Positive weight**
 else
 $B[j] \leftarrow 0$ **#Negative weight**
 end
return Array $B[1 \dots b]$

weighted binary strings of each feature word are added together to create a weight vector of the input data, and the weight vector that was created after merging is processed so that bits greater than 0 are set to 1 and bits less than 0 to 0.

To better reflect the advantages of the IIoT-Simhash algorithm, we have briefly illustrated the examples in Table II. We can see that Text1 and Text2 are highly similar in overall format and meaning, and there is only a slight difference in numeric data. The final calculation of similarity between Text1 and Text2 based on traditional Simhash is only 0.796875, which deviates from the actual scenario. Due to the fact that the traditional Simhash algorithm ignores the meaning of numeric data itself and treats all feature values as character-based data (including “36.9” and “37.1”), even the hash values that are produced have a significant disparity. The IIoT-Simhash algorithm, in contrast, better preserves the fine distinction between the numerical feature values, and the final judgment is accurate.

C. IIoT-SBSD: The Similarity-Based Secure Deduplication for IIoT

The IIoT-SBSD is made up of six polynomial-time algorithms (TAG, SimTest, FE-KG, FE-REP, ENC, DEC):

- 1) $\text{TAG}(1^\lambda, r_1, w) \rightarrow t_w$: This is a tag generation algorithm that uses the IIoT-Simhash function. The data owner runs this algorithm to calculate the abstract of the input data; the security parameter λ , random numbers $r_1 \in \{0, 1\}^\lambda$, and data w are utilized as input to return the data tags t_w .
- 2) $\text{SimTest}(t_w, \{T\}, d) \rightarrow 0/1$: After the data owner sends the data tag t_w to the edge node, the latter need to check the duplicate data by comparing data tags. If there exists node duplicate data, the edge node calculates the Hamming distance $\text{dis}_{Ham}(t_w, t_{w'})$ between t_w and $t_{w'} \in \{T\}$, and compares $\text{dis}_{Ham}(t_w, t_{w'})$ with a preset trapdoor d . Only

when $\text{dis}_{Ham}(t_w, t_{w'}) < d$, the algorithm returns 1, otherwise, returns 0.

- 3) $\text{FE} - \text{KG}(1^\lambda, r_2, t_w) \rightarrow \langle k_w, pk \rangle$: This is a fuzzy extraction-based key generation algorithm that allows the data owner to generate the encryption key for the data; with the security parameter λ , a random number $r_2 \in \{0, 1\}^\lambda$ and the data tag t_w as input, a random string encryption key k_w and auxiliary information P_w are output, where $k_w = H_1(R_w)$, H_1 is a mapping function as defined in Section IV-D, and $pk = g^{k_w}$.
- 4) $\text{FE} - \text{REP}(r_2, t_{w'}, P_w) \rightarrow \langle k_w, pk \rangle$: This represents a key regeneration algorithm designed for the data owner. By using the auxiliary parameters P_w and the fuzzy abstract of the data $t_{w'}$ as inputs, the private key k_w is only output when the Hamming distance $\text{dis}_{Ham}(t_{w'}, t_w)$ between $t_{w'}$ and t_w is smaller than a preset trapdoor d , denoted as $\text{dis}_{Ham}(t_{w'}, t_w) < d$; otherwise, output a random number.
- 5) $\text{ENC}(r_1, k_w, w) \rightarrow c_w$: This represents an encryption algorithm designed for the data owner. The specific encryption algorithm is $c_w = \text{ENC}(k_w, (w \oplus G(k_w, r_1)))$; where $G() \rightarrow \{0, 1\}^{|w|}$ is a pseudorandom generator that outputs a pseudo-random encryption key $G()$ of length $|w|$.
- 6) $\text{DEC}(r_1, k_w, c_w) \rightarrow w$: This is a decryption algorithm run by the data owner to compute the plaintext of the input data; it uses a ciphertext c_w and a private key k_w as input and outputs a plaintext $w = \text{DEC}(k_w, c_w) \oplus G(k_w, r_1)$.

The detailed algorithm is shown in Algorithm 2, where r_1 and r_2 are obtained through broadcast channel [35]. First, compute the data tag of the input data w by tag generation algorithm $\text{TAG}(1^\lambda, r_1, w) \rightarrow t_w$ and send it to nearly edge node for similarity comparison. If the edge node already has similar data, it returns the key auxiliary parameters P_w to the uploader. The uploader computer the key by key regeneration algorithm $\text{FE} - \text{REP}(r_2, t_{w'}, P_w) \rightarrow \langle k_w, pk \rangle$; if the edge node doesn't have similar data, the uploader computes the key by key generation algorithm $\text{FE} - \text{KG}(1^\lambda, r_2, t_w) \rightarrow \langle k_w, pk \rangle$. The uploader uses k_w to encrypt data w by encryption algorithm $\text{ENC}(k_w, t_w, w) \rightarrow c_w$ and uploads t_w, c_w, P_w to edge node. For w is general data, the edge node performs deduplication on w ; else the edge node stores w .

D. S-PoWs: Similarity-Based Proofs of Ownership

When a data analyst needs to query data, the CSP needs to authenticate the ownership of the user to ensure data security. Since our system is similarity-based deduplication, the existing PoWs [13] cannot be applied to our scheme. For this reason, we propose similarity-based proofs of ownership (S-PoWs) to implement data ownership verification. On input of the public parameter PP , encrypted data c_w , and the authenticator δ , this algorithm outputs the S-PoWs verification result.

Definition IV.1: Our S-PoWs contain the following five main steps.

- 1) $\text{Setup}(1^\lambda) \rightarrow \text{Para}$: This algorithm uses a security parameter λ as input, and generate a public parameter Para .

Algorithm 2: IIoT-SBSD(w).

Input: $w \in W$
Output: The ciphertext dataset C'_w of the similarity deduplicated dataset W'
 Upload $TAG(1^\lambda, r_1, w) \rightarrow t_w$ to Edge Node;

if $t_w \approx t_i, t_i \in T(u)$ **then**
 $FE-REP(r_2, t_w, P_w) \rightarrow \langle k_w, pk \rangle$ **#Key regeneration**
 $ENC(k_w, t_w, w) \rightarrow c_w$
else
 $FE-KG(1^\lambda, r_2, t_w) \rightarrow \langle k_w, pk \rangle$ **#Key generation**
 $ENC(k_w, t_w, w) \rightarrow c_w$
 Upload t_w, c_w, p_w to Edge Node;
for w is General Data **do**
 if $t_w \approx t_i, t_i \in T(u)$ **then**
 Deduplicate w
 else
 Storage c_w to C'_w
end
return C'_w

First, two multiplicative cyclic groups G_1 and G_2 with order p are used to construct a bilinear map $e : G_1 \times G_1 \rightarrow G_2$. After that, an element u and a generator g are selected from G_1 , two hash functions $H_1 : \{0, 1\}^* \rightarrow Z_p^*$ and $H_2 : \{0, 1\}^* \rightarrow G_1$ are used to generate the necessary metadata. To help the user or CSP produce the index and coefficient of challenged data, pseudo-random functions $\alpha : \{1, 2, \dots, n\} \times Z_p^* \rightarrow Z_p^*$ are defined. Additionally, a standard similarity-based proofs of ownership approach is developed to make it possible for data users to access the similarity data. Finally, the parameters of system can be calculated as $Para = \{G_1, G_2, e, p, g, u, H_1, H_2, \alpha\}$.

- 2) $AuthGen(k_w, pk) \rightarrow \delta$: This is a authentication algorithm run by the data owner; it takes a Key pairs $\langle k_w, pk \rangle$ as inputs and returns the authenticator δ . δ is calculated as follows.

$$\begin{cases} k_1 = H_1(k_w) \\ k_2 = H_2(pk) \\ T = H_1(k_1 || pk) \\ \delta = (k_2 u^T)^{k_w} \end{cases} \quad (1)$$

- 3) $Chal_{PoW}(Para) \rightarrow Chal$: takes $Para$ as input, and returns a PoW challenge $Chal = (z, r, P)$ as shown in (2), where z and r are random numbers, P is the auxiliary parameter of k_w . The CSP sends $Chal = (z, r, P)$ to data analyst.

$$\begin{cases} z \in [1, n] \\ r \in Z_p^* \\ Chal_{PoW} = (z, r, P) \end{cases} \quad (2)$$

- 4) $Proof_{PoW}(Chal, r_2, t_w) \rightarrow Proof$: To meet the challenge, data analyst calculates $a_i = \alpha(i, r)$ for each $r \in [1, z]$ and returns a proof $Proof$. The $Proof$ is calculated

as by (3).

$$\begin{cases} \langle k_w, pk \rangle = FE-REP(r_2, t_w, P) \\ k_1 = H_1(k_w) \\ \phi_i = a_i H_1(k_1 || pk) \\ Proof = \sum_{i=1}^z \phi_i \end{cases} \quad i \in [1, z] \quad (3)$$

- 5) $Verify_{PoW}(\delta, Chal, Proof) \rightarrow 0/1$: This algorithm takes δ and $Proof$ as inputs, and returns the verification result. CSP uses (4) to verify whether the challenge has passed.

$$e\left(\prod_{i=1}^z \delta^{a_i}, g\right) = e(k_2 u^{Proof}, pk) \quad (4)$$

Proof: We prove our proposed S-PoWs by a specific derivation of the formula, as shown below.

$$\begin{aligned} & e\left(\prod_{i=1}^z \delta^{a_i}, g\right) \\ &= e\left(\prod_{i=1}^z (k_2 u^T)^{k_w a_i}, g\right) \\ &= e\left(\prod_{i=1}^z (k_2 u^{H_1(k_1 || pk)})^{a_i}, g^{k_w}\right) \\ &= e\left(k_2 \prod_{i=1}^z (u^{\phi_i}), g^{k_w}\right) \\ &= e\left(k_2 u^{\sum_{i=1}^z \phi_i}, g^{k_w}\right) \\ &= e(k_2 u^{Proof}, pk) \end{aligned} \quad (5)$$

We can infer that the S-PoWs verification will pass as long as the data analyst has the encryption key k_w of the query data. In our system, only the data owner who has similar data can generate the same encryption key. Therefore, our proposed S-PoWs can realize similarity-based proofs of ownership.

E. Security Model

In order to formally the security of the S-PoWs, we define the security model. We define one *Game* interact between an adversary A and a challenger C to capture the S-PoWs. This game includes the following four phases.

- 1) *Set Up*: Challenger C randomly chooses a security parameter λ as input, and runs $Setup(1^\lambda) \rightarrow Para$ algorithm to generate public parameter $Para$.
- 2) *Query*: A selects and sends a set of data $CT = (c_1, \dots, c_i)$ as his inquiries to C adaptively. Then C responses the corresponding authenticators δ_i , public key pk_i and Hash of k_{w_i} to A until the query times up to q .
- 3) *Challenge*: In this phase, C submits the challenge $Chal'$ of data that have not been queried to A .
- 4) *Forgery*: Then, A run $Proof_{PoW}(Chal', r', t'_w) \rightarrow Proof'$ to generate a corresponding $Proof'$ according to $Chal'$, and then returns it to C .

- 5) *Win*: C Check whether the $Proof'$ can pass the S-PoWs. If proof $Proof'$ passes the verification algorithm $Verfiy$, that is $Verifi_{PoW}(\delta, Chal', Proof') \rightarrow 1$, then the adversary is the winner.

The following two major categories of attackers are taken into account in the framework of this paper:

- 1) *Internal attacker*: An adversary inside the system, typically the CSP and edge nodes. We assume that the edge nodes and the CSP are honest but curious, and believe that they can have arbitrary access to the data stored in them. In this paper, we argue that the CSP and a few edge nodes may collude to obtain personal data.
- 2) *External attacker*: An adversary external to the system, mainly referring to the malicious data owner. The external attacker attempts to obtain plaintext information by eavesdropping on the public channel to gather pertinent information. We define such adversaries as polynomial-time attackers A .

V. SECURITY ANALYSIS

In this part, we prove the security of the proposed scheme from the following segments: the security of S-PoWs, the security of the encryption key, resist of brute-force attacks, and the integrity of data on CSP.

Theorem 1 (The security of S-PoWs): There is no PPT adversary A that can pass the S-PoWs in the random oracle model as long as the underlying signature scheme utilized for authenticators in the IIoT-SBSD scheme is existentially unforgeable (DL and CDH problems are difficult in the bilinear group).

Proof: We prove this theorem in a series of games.

Game 0: It is a challenge game defined in Section IV-E.

Game 1: This game is the same as *Game 0*, and just one difference is that challenger C maintains a data tag list of all issued data. C aborts and states failure if the adversary A provides a tag that does not exist in the data tag list.

Game 2: This game is the same as *Game 1*, except that C keeps a list of its responses to authenticators δ and Hash of k_{w_i} query made by A . Utilizing this list, C observes each instance of S-PoWs protocol. If A pass S-PoWs but his $Proof'$ is not equal to $\sum_{i=1}^{Chal} \phi_i$ (where $Chal$ is the challenge issued by the C), C aborts and states failure.

Additionally, we are going to give a few important concepts for analyzing the differences in success probability between *Game 1* and *Game 2*. First, we suppose that the data resulting in the failure has an authenticator δ issued by the data owner and a generating element $g \in G_1$. Additionally, suppose that $Chal = (i, a_i)$ is the fails query and C returned ϕ'_i along with δ'_t in response to $Chal$. Let the expected response generated by a honest prover be ϕ_i and δ_t , where $\phi_i = a_i H_1(k_1 || pk)$ and $\delta = (k_2 u^T)^{k_w}$. Considering the correctness of the proposed scheme, the expected response satisfies the verification equation, i.e., that

$$e\left(\prod_{i=1}^z \delta^{a_i}, g\right) = e\left(\prod_{i=1}^z (H_2(pk) u^{\phi_i}), g\right) \quad (6)$$

From the failure stated by C , we know that $\delta'_t \neq \delta_t$ but δ'_t also satisfies the verification equation, i.e., that

$$e\left(\prod_{i=1}^z \delta'^{a_i}, g\right) = e\left(\prod_{i=1}^z (H_2(pk) u^{\phi'_i}), g\right) \quad (7)$$

where the public key $pk = g^{k_w}$. Observe that if $\phi_i = \phi'_i$ for all i , the verification equation indicates that $\delta'_t = \delta_t$, which goes against what we assumed earlier. Therefore, if we assume that $\Delta\phi_i = \phi'_i - \phi_i$ for all $i \in Chal$, then at least one of $\Delta\phi_i$ is nonzero. Based on this finding, we further prove whether there is a non-negligible difference between the success probability of A in *Game 1* and *Game 2* by constructing a simulator S to solve the CDH problem.

The simulator S takes $g, g^{k_w}, h \in G$ as inputs and tries to output h^{k_w} . Specifically, S behaves same as C in *Game 1*, the differences are as follow:

- 1) As instead of g^{k_w} as obtained in the challenge, S sets the public key pk during Set Up, indicating that S is unaware of the associated secret key k_w .
- 2) S models the hash function H_2 as a random oracle that keeps a list of queries and responses to answer consistently. In a query instance, it chooses a random $r \leftarrow Z_p^*$ and responds with $g^r \in G$.
- 3) When asked to store encrypted data, S behaves as below. First, S selects random values $\alpha, \beta \leftarrow Z_p^*$, such that $u = g^\alpha h^\beta$. After that, S keeps interacting with A up until the situation of *Game 2* occurs: Different from the expected authenticator δ_t , A passes the verification with δ'_t . We can calculate the following equation:

$$\begin{aligned} & e\left(\prod \delta'^{a_i}_t / \delta^{a_i}_t, g\right) \\ &= e\left(\prod (u^{\Delta\phi_i}), pk\right) \\ &= e\left(\prod ((g^\alpha h^\beta)^{\Delta\phi_i}), pk\right) \end{aligned} \quad (8)$$

Take it a step further, we obtain:

$$e\left(\prod \delta'^{a_i}_t / \delta^{a_i}_t \cdot pk^{-\sum \alpha \Delta\phi_i}, g\right) = e(h, pk)^{\sum \beta \Delta\phi_i} \quad (9)$$

We can now see that the CDH problem has an answer, that,

$$h^{k_w} = \left(\prod \delta'^{a_i}_t / \delta^{a_i}_t \cdot pk^{-\sum \alpha \Delta\phi_i}\right)^{\sum \frac{1}{\beta \Delta\phi_i}} \quad (10)$$

Therefore, we can build a simulator S to use A to solve the CDH problem if there is a non-negligible difference between the success probability of A in *Game 1* and *Game 2*.

Game 3. This game is the same as *Game 2*, with the exception that C monitors interactive instances and tracks δ queries. This time, we need to verify whether exist any of these instances A passes the verification but at least one of the aggregate messages ϕ_i is not equal to the expected $a_i H_1(k_1 || pk)$. Based on the same notations setting as *Game 2*, if the difference between the success probability of A in *Game 2* and *Game 3* is non-negligible, we can construct a simulator S to solve the DL problem.

Here, S takes $g, h \in G$ as inputs and aims to output x such that $h = g^x$. Specifically, S behaves like C in *Game 2*, the differences are as following:

- 1) S acts as follows when asked to store encrypted data. First, S selects random values $\alpha, \beta \leftarrow Z_p^*$, such that $u = g^{\alpha} h^{\beta}$.
- 2) Then, S keeps interacting with A until the special condition of *Game 3* occurs: A passes the verification with ϕ'_i and not the expected authorizer ϕ_i . So we can get that $\delta'_t = \delta_t$ and further calculate that:

$$e(\delta'_t, g) = e(\delta_t, g) \quad (11)$$

$$e\left(\prod \delta'^{a_i}_t / \delta^{a_i}_t, g\right) = e\left(\prod ((u)^{\Delta \phi_i}), pk\right) \quad (12)$$

$$1 = \prod (u)^{\Delta \phi_i} = g^{\sum \alpha \Delta \phi_i} \cdot h^{\sum \beta \Delta \phi_i} \quad (13)$$

Now, we have found the solution to the DL problem,

$$h = g^{-\frac{\sum \alpha \Delta \phi_i}{\sum \beta \Delta \phi_i}} \quad (14)$$

Therefore, we can build a simulator S and utilize A to solve the DL problem, if there is a non-negligible difference between the success probability of A in *Game 2* and *Game 3*.

Definition V.1 (The security of the key [36]): In a passive adversary attack, a fuzzy extractor $(M, m, l, t, \varepsilon)$ made up of a random extractor EXT of average degree and a security probability $\langle SS, REC \rangle$ of (M, m, m', t) , if adversary A obtains P_w , the output $k_w = R_w$ is still extremely close to the uniform distribution ε , that is, if $GEN(w) = \langle R, P \rangle$ then $dis((R, P), (U_1, P)) \leq \varepsilon$. In other words, if adversary A cannot discover information about w , it still cannot obtain the private key k_w even if it has access to the auxiliary parameters P_w .

Theorem 2 (The security of the encryption key): The IIoT-SBSD could be effective against external attacks. In other words, the probability of adversary A successfully gaining the plaintext is negligible if the original data w is unavailable but the data tag t_w is known.

$$\Pr[A_{wins}] \leq \varepsilon. \quad (15)$$

Proof: According to the system's design, if adversary A wants to get the data w , it must communicate with the CSP. According to Theorem 1, the probability that adversary A can pass S-PoWs authentication with only t_w is negligible. Therefore, the probability of adversary A winning the $Game_{S-PoWs}$ by obtaining the c_w with the t_w alone is negligible, $\Pr[A_{wins}] \leq \varepsilon$.

According to the decryption algorithm $DEC(r_1, k_w, c_w) \rightarrow w$, if adversary A wants to obtain plaintext information through ciphertext c_w , adversary A needs to obtain k_w and r_1 . In our design, the k_w can be generated by the key regeneration algorithm without being stored on any entity. That is to say, if adversary A has obtained c_w , the only way to decrypt c_w is to regenerate the key using $FE - REP(r_2, t_w, P_w) \rightarrow k_w$. Without P_w , adversary A is still unable to recover the k_w and decrypt c_w to obtain the w . Even if adversary A may compromise the edge node or CSP and obtains the auxiliary parameter P_w , it still cannot generate k_w due to the lack of r_2 . However, according to our design, only the data owner who has the w could obtain r_1 and r_2 through the broadcast channel. This means that the probability of adversary A winning the *Game* by decrypting c_w to get the

w is negligible, $\Pr[A_{wins}] \leq \varepsilon$. Therefore, the IIoT-SBSD can efficiently fend off external attacks and guarantee data security.

Theorem 3 (Resisting brute-force attacks): Given a plaintext space W and a set of encrypted $\{c_{w_1}, \dots, c_{w_i}, \dots\}$, the attacker A can not determine whether $c_{w_i} = c_{w'}$ by selecting a random data $w' \in W$ in an exhaustive way.

Proof: To show that our design is able to resist brute-force attacks, we use the security game for semantic security [37] in the IIoT-Simhash, and set its security to the DL (Diffie-Hellman) assumption, i.e., our proposed scheme is able to resist brute-force attacks only if an external attacker, edge node, or CSP has no access to the secret key. In addition, even if the plaintext space is large enough, it still can hardly perform brute-force attacks [38] successfully. This paper only presents a succinct security analysis.

In this subsection, we mainly consider a brute-force attack launched by an external attacker A . Since the external attacker A does not have any background knowledge background, assuming that the edge node or CSP is allied with A , the external attacker A can only obtain the data similarity label t_w and the data ciphertext c_w and its auxiliary information P_w . In particular, A first attempts to compute the data key $FE - REP(r_2, t_w, P_w) \rightarrow k_w$. However, since A can hardly infer the valid data w , it cannot obtain the auxiliary parameters r_2 via broadcast. Although A launches the scale analysis attack [39] such that $c_{w_i} = c_{w_j}$, it just deduces the equality information (e.g., $w_i = w_j$), without outputting a guess $w = w'$, $w' \in W$. Thus, A can hardly launch a successful chosen-ciphertext attack.

For the same reason, the edge node and CSP also cannot launch a chosen-ciphertext attack successfully without obtaining the auxiliary parameters r_1 and r_2 .

Theorem 4 (The integrity of data on CSP): The data owner can verify the integrity of the data by checking the consistency of the tags during the data download phase.

The integrity is based on the tag's uniqueness. For uploading data, our system could resist forgery attacks by malicious data owners and effectively prevents compromising data integrity by uploading fake data. When a malicious data owner unites the local server as an edge node and uploads a ciphertext that does not match the tag, the CSP can detect the problem by comparing it with the existing data. Even if the malicious data owner is the initial uploader and generates the t_w of w , the CSP finds the anomaly through data comparison during the subsequent data uploads, without affecting the availability of the data to the following data owner. Although the CSP cannot immediately distinguish whether the current data is fake, with more subsequent data owners uploading similar data, the CSP can determine by the number of copies of a particular file and log it, which effectively detects forgery attacks by malicious data owners.

Specifically, CSP will check both the consistency of t_w, c_w , and the consistency of $t_w, t_i \in \{T\}$. Apparently, the consistency of t_w and c_w is based on the collision probability of the hash function, and this probability is negligible in the proposed system.

Proof: Suppose adversary A stores the ciphertext c'_w in the CSP along with the inconsistent tag t_w by tampering with the w . When the data owner initiates a data query, the legitimate data

TABLE III
DATASETS

Dataset	Alias	Type	Records
Server Logs	Dataset1	Real	100000
Telecom IOT Dataset	Dataset2	Real	96478

owner can obtain the data ciphertext c_w^* by tag query on the CSP. Once the data owner has obtained the ciphertext, the data integrity can be verified by the following steps.

- Calculate the k_{w^*} by the key regeneration algorithm $FE - REP(r_2, t_w, P_w) \rightarrow k_{w^*}$;
- Calculate the w^* by the decryption algorithm $DEC(r_1, k_{w^*}, c_{w^*}) \rightarrow w^*$;
- Calculate the t_w^* of the w^* by the tag generation algorithm $TAG(1^\lambda, r_1, w) \rightarrow t_{w^*}$;
- Compare whether $t_w = t_{w^*}$.

Since w has been tampered with, $t_w \neq t_{w^*}$, proving that data integrity has been compromised. Therefore, the data owner can verify the integrity of the data by checking the consistency of the tags during the data download phase.

VI. PERFORMANCE EVALUATION

In this section, we evaluate various performances in IIoT-SBSD and compare them with existing approaches [40] and [12]. We evaluate the performance of the IIoT-SBSD framework under two different scenarios. To simulate real IIoT scenarios as much as possible, the datasets used in our experiments are the real datasets named *Server Logs* and *Telecom IOT Datasets* [41], as shown in Table III. The simulations were performed in Python 3.10 on a desktop computer with Intel Core i7-9700 CPU and 64 G RAM running Windows 10. Additionally, We investigate the effect of deduplication in several nodes by constructing virtual machines in the computer to simulate it. We run multiple processes to simulate large-scale cluster environments in the constructed simulated environment. We also average the results of 100 experiments under the same environment as the final experimental result to ensure its reliability.

A. Experiment 1: The Efficiency of IIoT-Simhash and Simhash

The IIoT-SBSD system proposed in this paper is mainly based on similarity tags for data similarity comparison. Therefore, we compare the computational overhead of the IIoT-Simhash algorithm with the traditional Simhash's, where the size of the tags is 64 bits, as shown in Fig. 5(a). It is clear that the IIoT-Simhash algorithm's computational overhead is similar to that of the traditional Simhash algorithm. In most IIoT scenes, the size of the generated data samples is less than 20 k, hence the processing overhead difference between the two is currently insignificant.

We also compare the data similarity S_i of the data tags (where $i \in [Simhash, IIoT - Simhash]$) with the plaintext similarity S_p to calculate the similarity comparison accuracy (Sca), which is calculated by the formula $Sca = 1 - \frac{|S_p - S_i|}{S_p}$, and the specific results are shown in Fig. 5(b). As observed in Fig. 5(b), the traditional Simhash has a significant mistake in data samples,

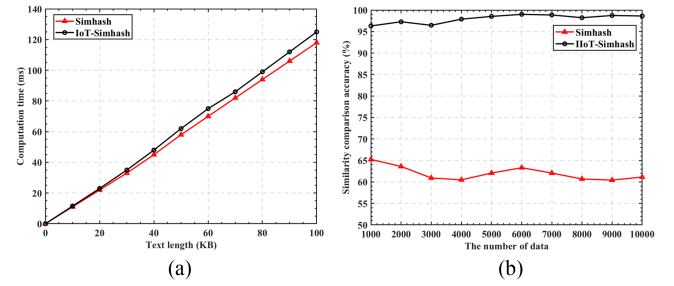


Fig. 5. The efficiency of IIoT-simhash and simhash.

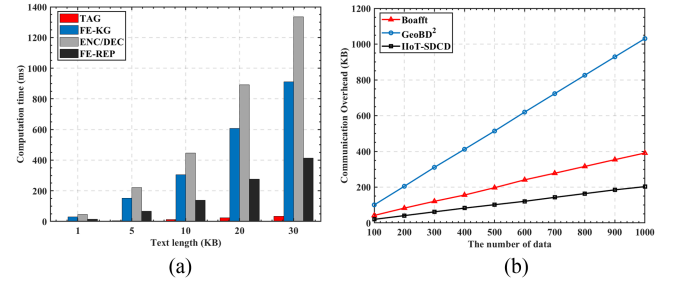


Fig. 6. Computing overhead and communication overhead.

and its similarity comparison accuracy is only approximately 60%. On the contrary, our improved IIoT-Simhash algorithm can better adapt to the data sample characteristics with an accuracy of over 95%.

B. Experiment 2: Computing Overhead and Bandwidth Overhead

We test the IIoT-SBSD system in terms of both computational overhead and communication overhead in our sample dataset.

Computing Overhead: We calculate the computational overhead of the main algorithms in the IIoT-SBSD system, mainly including the tag generation algorithm, key generation algorithm, encryption, and decryption algorithm. The results are shown in Fig. 6(a), where the vertical coordinate is the computing time overhead and the horizontal coordinate is the data size. Fig. 6(a) is obvious that the computation time of these four algorithms is linearly related to the data size. Compared to other techniques, the tag generation time cost is also small and negligible. This shows that the IIoT-Simhash method we improved can handle the computing needs of millions of industrial IoT devices without adding to the system's overall computational overhead.

Communication Overhead: Fig. 6(b) shows the communication overhead incurred for data uploading after different deduplication operations. Since cloud deduplication requires all data to be uploaded to the cloud first, so the *GeoBD²* scheme costs the most communication overhead. The *Boafft* scheme allows the data owner to remove the duplicate data directly locally, which directly eliminates all duplicate data uploads and therefore has a lower communication overhead. Different from them, our approach employs edge nodes to carry out data deduplication operations, and only uploads the deduplicated

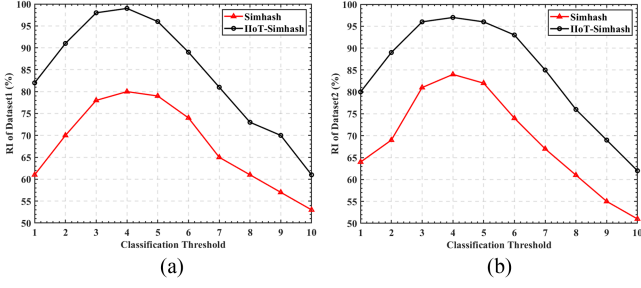


Fig. 7. The classification accuracy.

data that is still valid to the CSP. Besides, our approach can also perform deduplication operations on similar data, which further saves communication overhead. The results show that our method can reduce communication overhead to the maximum extent compared to the other two schemes.

C. Experiment 3: The Classification Accuracy

In this experiment, we determine the classification accuracy based on IIoT-Simhash and Simhash by using the industry accepted Rand Index (RI). The RI is determined from the following four components using the permutation concept:

TP: refers to the number of instances where two same samples are classified together.

FP: represents the number of instances in which two non-identical samples are split into the same cluster.

TN: represents the number of instances in which two non-identical samples are split into different clusters.

FN: refers to the number of instances where two same samples are split into two clusters.

The specific formula for RI is :

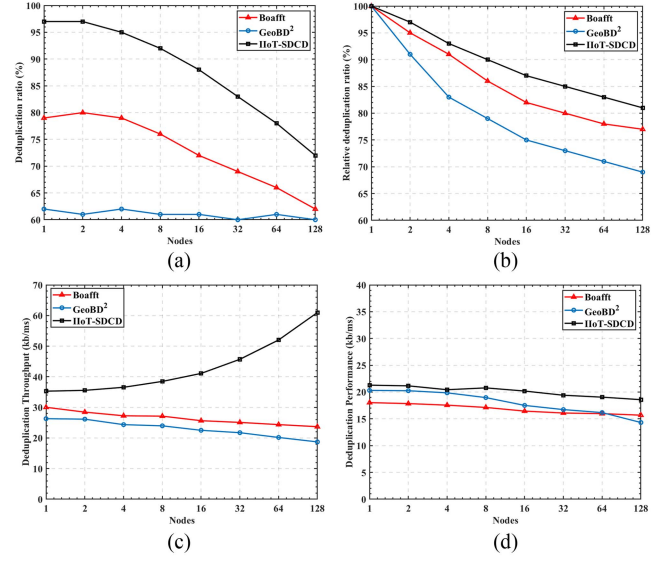
$$RI = \frac{TP + TN}{TP + FP + FN + TN}. \quad (16)$$

We can deduce from the formula that RI has a value between $[0, 1]$. The clustering effect is improved with a bigger RI value.

We cluster data using the similarity-preserving tag generation algorithms Simhash and IIoT-Simhash, and we evaluate the clustering effect by determining the RI value of the clustering. Because the classification threshold also significantly influences the clustering outcomes, various threshold sizes are chosen to observe the changes in RI values, and the results are displayed in Fig. 7. When classification threshold $T = [3, 4, 5]$, as can be shown, both techniques produce better clustering results. Additionally, our proposed IIoT-Simhash algorithm has a superior overall clustering effect than the traditional Simhash, which indicates that the IIoT-Simhash algorithm is more suitable for IIoT data clustering. Compared to the traditional Simhash, the enhanced IIoT-Simhash algorithm significantly improves the optimal clustering effect on the sample data by around 20%.

D. Experiment 4: Deduplication Ratio

We test the deduplication ratio of *Boafft* [40], *GeoBD²* [19], and our IIoT-SBSD for different edge node cases. We

Fig. 8. Deduplication ratio of dataset1 among *Boafft* [40], *GeoBD²* [19] and our IIoT-SBSD.

evaluate and compare the three systems using the deduplication ratio, relative deduplication ratio, deduplication throughput, and deduplication performance. Deduplication ratio (DR) represents the ratio of data's logical size to its physical size. Relative deduplication ratio (RDR) represents the ratio of DR gained by similarity matching within an edge node to that of DRs gained by global matching. Deduplication throughput (DT) is the ratio of the original data size to the time for uploading data. Deduplication performance (DP) represents the size of data deleted by deduplication per unit of time.

We simulate the four experiments mentioned above on two distinct datasets and compared the IIoT-SBSD system to the *Boafft* and *GeoBD²* systems. The findings are displayed in Figs. 8 and 9, respectively. Figs. 8(a) and 9(a) show the deduplication ratio of the three systems for different node sizes. The deduplication ratio in both images shows the same pattern of variation, and both indicate that the deduplication ratio of IIoT-SBSD is larger than the others. The reason is that IIoT-SBSD could perform similar-based deduplication. Therefore, compared with *Boafft* and *GeoBD²*, IIoT-SBSD has more advantages in IIoT data scenarios.

Figs. 8(b) and 9(b) show the results of the relative deduplication ratio for the three schemes with different node sizes. The data show that when the number of nodes is 1, the RDR always equals 1. And, as the number of nodes rises, the RDR of all three methods gradually declines. The above results must exist because the increase of available storage nodes makes the data distribution more dispersed, which in turn makes the proportion of similar data per node decrease. Due to its inability to deduplicate similar data, *GeoBD²* has a lower deduplication ratio than the others. When the cluster includes 128 nodes, *Boafft*'s deduplication ratio remains at about 80%. In a large-scale cloud storage-based deduplication system, the local similarity routing technique used in *Boafft* can somewhat improve RDR.

TABLE IV
COMPARISON OF FUNCTIONALITY

Scheme	Similarity comparison	Deduplication level	Key security	Data security	Data integrity	Availability
[18]	—	Block level	✓	✓	✓	✓
[20]	—	File level	✓	✓	—	✓
[42]	—	File level	✓	✓	—	✓
[43]	—	File level	✓	✓	—	—
[19]	Ciphertext	File level	—	—	—	—
[22]	Ciphertext	Block level	✓	✓	—	✓
Ours	Ciphertext	File level	✓	✓	✓	✓

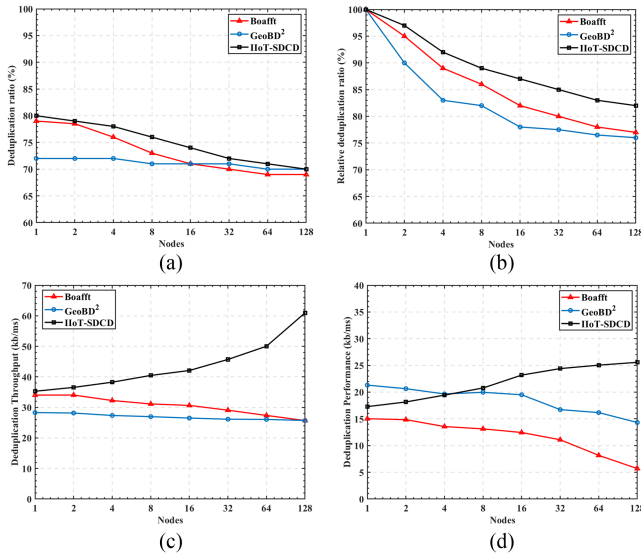


Fig. 9. Deduplication ratio of dataset2 among *Boafft* [40], *GeoBD²* [19] and our IIoT-SBSCD.

In comparison, our IIoT-SBSCD scheme performs well in the deduplication ratio. When the number of nodes in the cluster reaches 128, we can obtain about 83% of RDR within a single node.

The experiments on the deduplication throughput and system performance of the three approaches at various node sizes are shown in Figs. 8(c), 9(c), and 8(d), 9(d). Overall, the deduplication throughput and system performance of IIoT-SBSCD are better. The main reason is that IIoT-SBSCD performs data deduplication at the edge nodes, and the operations within each node are parallelizable. As the number of nodes rises, more parallel operations boost the deduplication efficiency per unit of time to a greater extent. So there is a significant increase in IIoT-SBSCD throughput after the number of nodes reaches 8 in Figs. 8(c) and 9(c). The IIoT-SBSCD technique only uploads the deduplicated data, reducing transmission overhead. *Boafft* and *GeoBD²*'s storage performance declines as the number of nodes rises. Although *Boafft*'s deduplication rate is not as high as IIoT-SBSCD's, it performs better on storage and uses less Memory space. Due to the fact that all deduplication operations of *GeoBD²* are done in the cloud, and its unique lookup method Geo-DHBT consumes more time as the number of nodes increases. This results in lower deduplication throughput and system performance than the other two solutions.

As can be seen from Fig. 8(d) and 9(d), the system performance of IIoT-SBSCD gradually smoothed out with the increase of nodes. This is because as the number of nodes increases, more deduplication operations can be performed in parallel per unit of time, so the system performance tends to increase with the increase of nodes. However, as the number of nodes increases, it makes the data distribution more dispersed and the proportion of similar data within a unit node decreases, tending therefore to cause the system performance to eventually level off.

E. Security Comparison

We now compare our scheme's security with [18], [19], [20], [22], [42], [43], all of them are the latest deduplication research from the last three years. We mainly focus on six aspects: similarity comparison, deduplication level, key security, data security, data integrity, and data availability, as shown in Table IV. The focus of our study differs from that of previous works ([18], [20], [42] and [43]); these techniques are designed for secure deduplication scheme, and the security of these solutions is well thought out. But, they do not consider the deduplicate need for similar data, making them not suitable for IIoT scenarios. Same as [19] and [22], our scheme focuses on similarity-based secure deduplication and realizes ciphertext similarity comparison, which enables quick comparison of data similarity while ensuring data security. Furthermore, our scheme and [22] both consider data security and design similarity-based proof of data ownership to ensure data availability after similar deduplication. However, data integrity is neglected in [22], making it unable to resist data tampering problems caused by CSP dishonesty. We emphasize data integrity in the similarity-based deduplication process that is similar to the scheme, and we also take data security into account. The proposed IIoT-SBSCD algorithm can achieve fast similar data comparison in the ciphertext and has higher accuracy than the traditional Simhash algorithm in IIoT scenarios. Compared to conventional deduplication techniques, our scheme significantly increases the availability of IIoT data while ensuring security and saving storage space wherever possible.

VII. SUMMARY AND FUTURE WORK

This paper proposes a similarity-based secure deduplication system for IIoT cloud management, which provides a practical data management approach for IIoT data analysis. In particular, we build a parallelizable edge-based deduplication framework, which performs the data deduplication at the edge nodes, and

allows these edge nodes to deliver only the deduplicated data to CSPs, thus greatly improving the data transmission efficiency. The proposed IIoT-SBSD method can directly measure the similarity of encrypted data and perform IIoT data deduplication on ciphertexts, significantly reducing storage costs while maintaining data security. In addition, to avoid the impact of deduplication on data availability, we design a similarity-based proofs of ownership, S-PoWs, to implement data ownership verification and further improve data security. Experiments demonstrate that our method significantly reduces storage space and transmission bandwidth without increasing computational overhead.

For future work, we plan to design a suitable data analysis scheme based on the proposed system. Further, we will also work on how to achieve decentralized IoT data storage management to meet the advent of the Web 3.0 era.

REFERENCES

- [1] B. Alturki and S. Reiff-Marganiec, "Towards an off-the-cloud IoT data processing architecture via a smart car parking example," in *Proc. 2nd Int. Conf. Internet Things Data Cloud Comput.*, 2017, pp. 1–5.
- [2] L. J. Trautman and M. T. Hussein, "The Internet of Things (IoT) in a post-pandemic world," Available at SSRN 4149477, 2022.
- [3] Y. Lu, X. Huang, Y. Dai, S. Maharjan, and Y. Zhang, "Blockchain and federated learning for privacy-preserved data sharing in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 6, pp. 4177–4186, Jun. 2019.
- [4] R. H. Weber, "Internet of things: Privacy issues revisited," *Comput. Law Secur. Rev.*, vol. 31, no. 5, pp. 618–627, 2015.
- [5] C. Wang, T. Zhou, J. Shen, W. Wang, and X. Zhou, "Searchable and secure edge pre-cache scheme for intelligent 6G wireless systems," *Future Gener. Comput. Syst.*, vol. 140, pp. 129–137, 2023.
- [6] T. Vijayakumar and R. Vinothkanna, "Retrieval of complex images using visual saliency guided cognitive classification," *J. Innov. Image Process.*, vol. 2, no. 2, pp. 102–109, 2020.
- [7] S. Venkatraman and R. Venkatraman, "Big Data security challenges and strategies," *AIMS Math.*, vol. 4, no. 3, pp. 860–879, 2019.
- [8] J. Li, J. Wu, G. Jiang, and T. Srikanthan, "Blockchain-based public auditing for Big Data in cloud storage," *Inf. Process. Manage.*, vol. 57, no. 6, 2020, Art. no. 102382.
- [9] P. Singh, N. Agarwal, and B. Raman, "Secure data deduplication using secret sharing schemes over cloud," *Future Gener. Comput. Syst.*, vol. 88, no. pp. 156–167, 2018.
- [10] Z. Yan, L. Zhang, D. Wenxiu, and Q. Zheng, "Heterogeneous data storage management with deduplication in cloud computing," *IEEE Trans. Big Data*, vol. 5, no. 3, pp. 393–407, Sep. 2017.
- [11] K. Vijayalakshmi and V. Jayalakshmi, "Shared access control models for Big Data: A perspective study and analysis," in *Proc. Int. Conf. Intell. Comput. Inf. Control Syst.*, 2021, pp. 397–410.
- [12] Z. Yan, M. Wang, Y. Li, and A. V. Vasilakos, "Encrypted data management with deduplication in cloud computing," *IEEE Cloud Comput.*, vol. 3, no. 2, pp. 28–35, Mar./Apr. 2016.
- [13] K. Huang, X.-S. Zhang, Y. Mu, F. Rezaeibagha, and X. Du, "Bidirectional and malleable proof-of-ownership for large file in cloud storage," *IEEE Trans. Cloud Comput.*, vol. 10, no. 4, pp. 2351–2365, Fourth Quarter 2021.
- [14] B. D. Deebak et al., "In the digital age of 5G networks: Seamless privacy-preserving authentication for cognitive-inspired internet of medical things," *IEEE Trans. Ind. Informat.*, vol. 18, no. 12, pp. 8916–8923, Dec. 2022.
- [15] P. Puzio, R. Molva, M. Önen, and S. Loureiro, "PerfectDedup: Secure data deduplication," in *Proc. Int. Workshop Quantitative Aspects Secur. Assurance*, 2015, pp. 150–166.
- [16] J. Li, T. Li, Z. Liu, and X. Chen, "Secure deduplication system with active key update and its application in IoT," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 6, pp. 1–21, 2019.
- [17] H. Yuan, X. Chen, J. Wang, J. Yuan, H. Yan, and W. Susilo, "Blockchain-based public auditing and secure deduplication with fair arbitration," *Inf. Sci.*, vol. 541, pp. 409–425, 2020.
- [18] G. Tian et al., "Blockchain-based secure deduplication and shared auditing in decentralized storage," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 3941–3954, Nov./Dec. 2022.
- [19] K. RabindraBarik, S. S. Patra, R. Patro, S. N. Mohanty, and A. A. Hamad, "GeoBD2: Geospatial Big Data deduplication scheme in fog assisted cloud computing environment," in *Proc. IEEE 8th Int. Conf. Comput. Sustain. Glob. Develop.*, 2021, pp. 35–41.
- [20] Y. Ming et al., "Blockchain-enabled efficient dynamic cross-domain deduplication in edge computing," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 15639–15656, Sep. 2022.
- [21] C. Zhang, Y. Miao, Q. Xie, Y. Guo, H. Du, and X. Jia, "Privacy-preserving deduplication of sensor compressed data in distributed fog computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4176–4191, Dec. 2022.
- [22] T. Jiang et al., "FuzzyDedup: Secure fuzzy deduplication for cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 3, pp. 2466–2483, May/Jun. 2023.
- [23] N. Jayapandian and A. M. J. Md Zubair Rahman, "Secure deduplication for cloud storage using interactive message-locked encryption with convergent encryption, to reduce storage space," *Braz. Arch. Biol. Technol.*, vol. 61, pp. 251–263, 2018.
- [24] M. Bellare, S. Kveedhi, and T. Ristenpart, "Message-locked encryption and secure deduplication," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2018, pp. 296–312.
- [25] B. Sudharsan, D. Sundaram, P. Patel, J. G. Breslin, and M. I. Ali, "Edge2Guard: Botnet attacks detecting offline models for resource-constrained IoT devices," in *Proc. IEEE Int. Conf. Pervasive Comput. Commun. Workshops Other Affiliated Events (PerCom Workshops)*, 2021, pp. 680–685.
- [26] L. Wang, B. Wang, W. Song, and Z. Zhang, "A key-sharing based secure deduplication scheme in cloud storage," *Inf. Sci.*, vol. 504, pp. 48–60, 2019.
- [27] Y. Wang, M. Miao, J. Wang, and X. Zhang, "Secure deduplication with efficient user revocation in cloud storage," *Comput. Standards Interfaces*, vol. 78, 2021, Art. no. 103523.
- [28] T. Zhang, J. Shen, C.-F. Lai, S. Ji, and Y. Ren, "Multi-server assisted data sharing supporting secure deduplication for metaverse healthcare systems," *Future Gener. Comput. Syst.*, vol. 140, pp. 299–310, 2023.
- [29] G. S. Manku, A. Jain, and A. D. Sarma, "Detecting near-duplicates for web crawling," in *Proc. 16th Int. Conf. World Wide Web*, 2007, pp. 141–150.
- [30] S. Sood and D. Loguinov, "Probabilistic near-duplicate detection using simhash," in *Proc. 20th ACM Int. Conf. Inf. Knowl. Manage.*, 2011, pp. 1117–1126.
- [31] S. Wu et al., "EaD: A collision-free and high performance deduplication scheme for flash storage systems," in *Proc. IEEE 38th Int. Conf. Comput. Des.*, 2020, pp. 155–162.
- [32] P. Talasila and D. E. Lucani, "Generalized deduplication: Lossless compression by clustering similar data," in *Proc. IEEE 8th Int. Conf. Cloud Netw.*, 2019, pp. 1–4.
- [33] J. Liu, J. Wang, X. Tao, and J. Shen, "Secure similarity-based cloud data deduplication in ubiquitous city," *Pervasive Mobile Comput.*, vol. 41, pp. 231–242, 2017.
- [34] Y. Dodis, L. Reyzin, and A. Smith, "Fuzzy extractors: How to generate strong keys from biometrics and other noisy data," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, 2004, pp. 523–540.
- [35] Q. Yu, J. Li, and S. Ji, "Anonymous identity based broadcast encryption against continual side channel attacks in the state partition model," *Appl. Sci.*, vol. 12, no. 18, 2022, Art. no. 9395.
- [36] Y. Wen, S. Liu, and S. Han, Reusable fuzzy extractor from the decisional Diffie–Hellman assumption," *Des. Codes Cryptogr.*, vol. 86, no. 11, pp. 2495–2512, 2018.
- [37] J. Stanek, A. Sorniotti, E. Androulaki, and L. Kencl, "A secure data deduplication scheme for cloud storage," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.*, 2014, pp. 99–118.
- [38] A. K. Agarwal, L. Rani, R. G. Tiwari, T. Sharma, and P. K. Sarangi, "Honey encryption: Fortification beyond the brute-force impediment," in *Advances in Mechanical Engineering*. Berlin, Germany: Springer, 2021, pp. 673–681.
- [39] M. Zhao et al., "Verifiable and privacy-preserving ranked multi-keyword search over outsourced data in clouds," in *Proc. IEEE 15th Int. Conf. Big Data Sci. Eng.*, 2021, pp. 95–102.
- [40] S. Luo, G. Zhang, C. Wu, U. S. Khan, and K. Li, "Boafft: Distributed deduplication for Big Data storage in the cloud," *IEEE Trans. Cloud Comput.*, vol. 8, no. 4, pp. 1199–1211, Fourth Quarter 2015.
- [41] Internet dataset, 2022. Accessed: Sep. 20, 2022. [Online]. Available: <https://www.kaggle.com/datasets>
- [42] X. Yang, R. Lu, J. Shao, X. Tang, and A. Ghorbani, "Achieving efficient secure deduplication with user-defined access control in cloud," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 591–606, Jan./Feb. 2022.

- [43] S. Jiang, J. Liu, Y. Zhou, and Y. Fang, "FVC-Dedup: A secure report deduplication scheme in a fog-assisted vehicular crowdsensing system," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2727–2740, Jul./Aug. 2022.



Yuan Gao (Graduate Student Member, IEEE) received the BS degree in artificial intelligence and MS degree in computer technology from Qingdao University, Qingdao, China, in 2017 and 2020, respectively. She is currently working toward the PhD degree with School of Cyber Science and Engineering, Southeast University, Nanjing, China. Her research interests include game theory based security solutions, information security, data deduplication, and Internet of Things (IoT) data security.



Shui Yu (Fellow, IEEE) received the PhD degree from Deakin University, Australia, in 2004. He currently is a professor with School of Computer Science, University of Technology Sydney, Australia. His research interest includes Big Data, Security and Privacy, Networking, and Mathematical Modelling. He has published three monographs and edited two books, more than 400 technical papers, including top journals and top conferences, such as *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Computers*, *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Emerging Topics in Computing*, *IEEE/ACM Transactions on Networking*, and *INFOCOM*. His h-index is 58. He initiated the research field of networking for Big Data, in 2013, and his research outputs have been widely adopted by industrial systems, such as Amazon cloud security. He is currently serving a number of prestigious editorial boards, including IEEE Communications Surveys and Tutorials (Area Editor), IEEE Communications Magazine, IEEE Internet of Things Journal, and so on. He is a member of AAAS and ACM, a distinguished lecturer of IEEE Communications Society, and an elected member of Board of Governor of IEEE Vehicular Technology Society.



Liquan Chen (Senior Member, IEEE) received the PhD degree from Southeast University, China, in 2005. He worked as a postdoc in Southeast University from 2005 to 2007, and an associate professor with Southeast University from 2008 to 2018. He worked as visiting scholar in the National University of Singapore, Singapore from 2011 to 2012. He now is a professor in School of Cyber Science and Engineering, Southeast University, Nanjing, China. His research interests include information security, cryptography and network security protocol, etc.



Huiyu Fang (Student Member, IEEE) received the BS degree in information engineering and MS degree in software engineering from Southeast University, Nanjing, China, in 2014 and 2018, respectively. He is currently working toward the PhD degree with the School of Cyber Science and Engineering, Southeast University, Nanjing, China. His research interests include privacy preserving, information security, and IoT security.



Jinguang Han (Senior Member, IEEE) received the PhD degree from University of Wollongong, Australia, in 2013. He is a professor with the School of Cyber Science and Engineering, Southeast University, China. He has published 50 papers in referred journal and conferences, including *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Computers*, *ESORICS*, etc. He served as a program committee co-chair

of provSec'16, FCS'19 and SPNCE'20, and a program committee member in several conferences, including SecureCom'21, ESORICS'20, PST'19, IS-PEC'18, CANS'17, TrustCom'15, etc. His research focuses on access control, blockchain, cryptography, cloud computing and privacy-preserving systems.